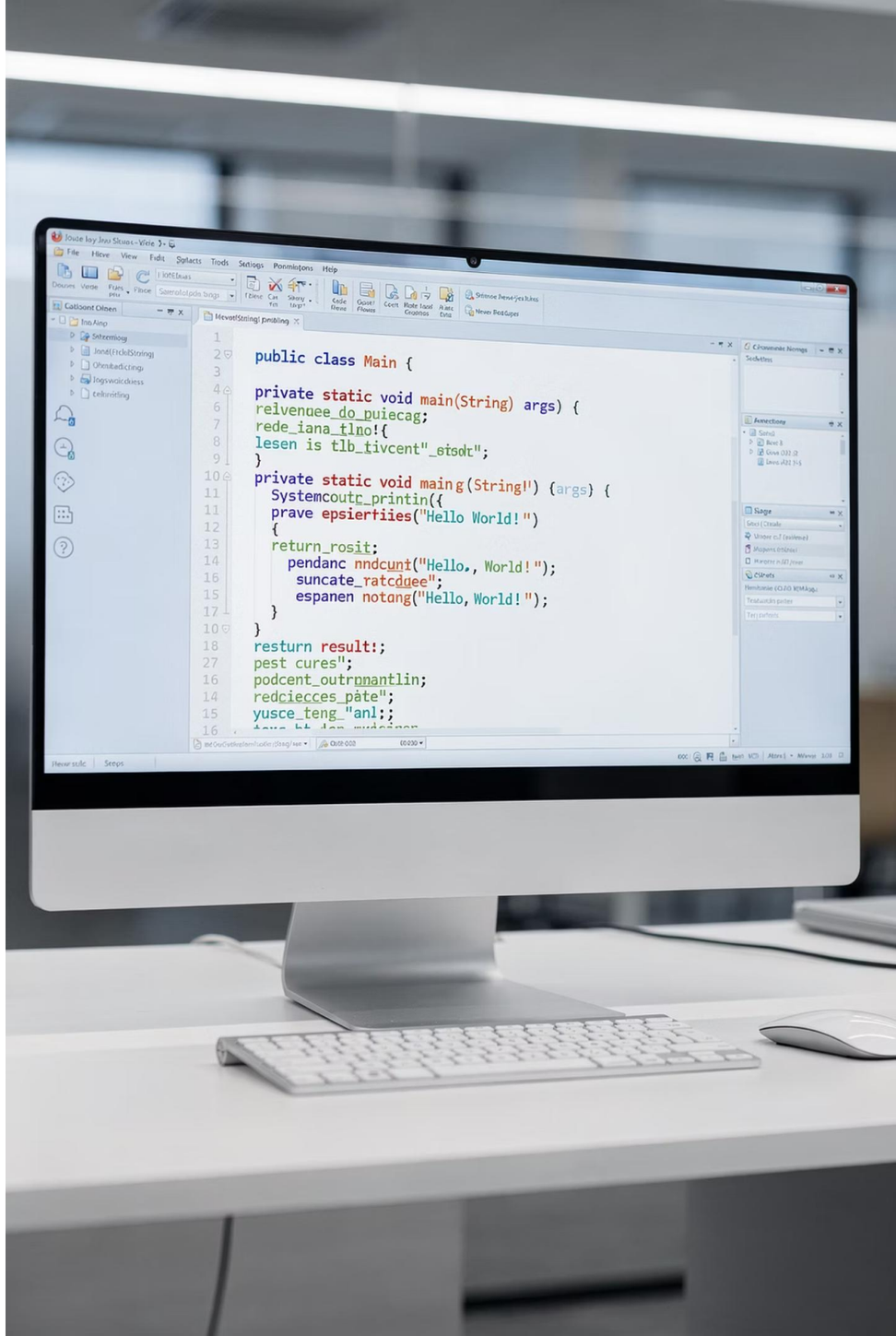
Module 5: GUI Programming with Java Swing

A comprehensive guide to building powerful graphical user interfaces using Java's Swing framework — covering event handling, MVC architecture, Swing components, and database connectivity.

CHAPTER 31



Module 5 Overview

What We'll Cover

01

Event Handling

Mechanisms, Event Classes, Sources of Events, and Listener Interfaces

02

Swing Key Features

Lightweight components, Pluggable Look and Feel, MVC architecture

03

Swing Controls

Components, Containers, Packages, and common UI elements

04

Event Handling in Swing

Delegation event model, ActionListener, ItemListener in practice

05

Database Connectivity

Connecting Swing applications to databases

CHAPTER 1

Introducing Swing

Swing is a framework providing more powerful and flexible GUI components than the original AWT. It has been the standard GUI toolkit for Java programmers for over a decade. Coverage spans three chapters: core concepts, common components, and Swing-based menus.

Note: For comprehensive coverage, see *Swing: A Beginner's Guide* by McGraw-Hill Professional (2007).

What is GUI?

- GUI = Graphical User Interface
- Examples: Calculator, ATM, Desktop Apps
- Contains windows, buttons, text fields, menus

CLI vs GUI

- CLI: Command line interaction
- GUI: Visual interface with buttons and fields
- GUI is easier for users

The Origins of Swing

The AWT Problem

The original AWT translated visual components into platform-specific **peers** (native code). This made components **heavyweight** and caused:

- Inconsistent look across platforms
- Fixed, unchangeable look and feel
- Restrictions (e.g., always opaque)

The Swing Solution

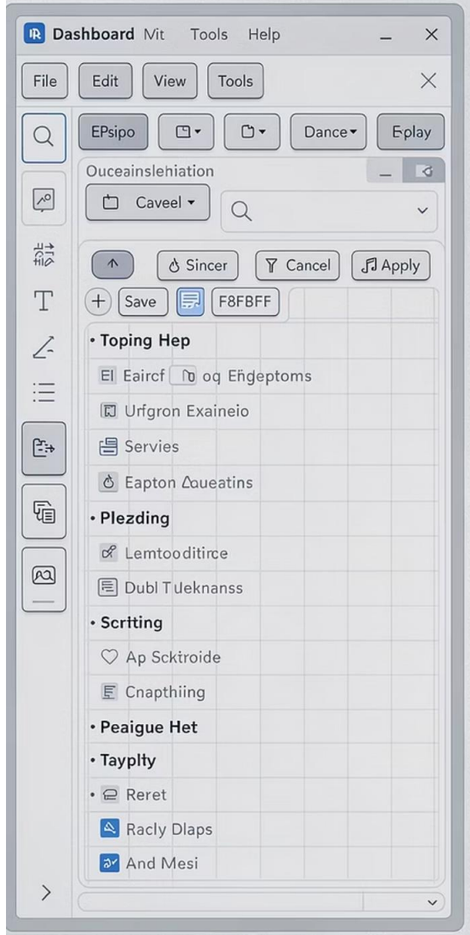
Introduced in **1997** as part of the Java Foundation Classes (JFC). Available as a separate library in Java 1.1, then **fully integrated in Java 1.2**.

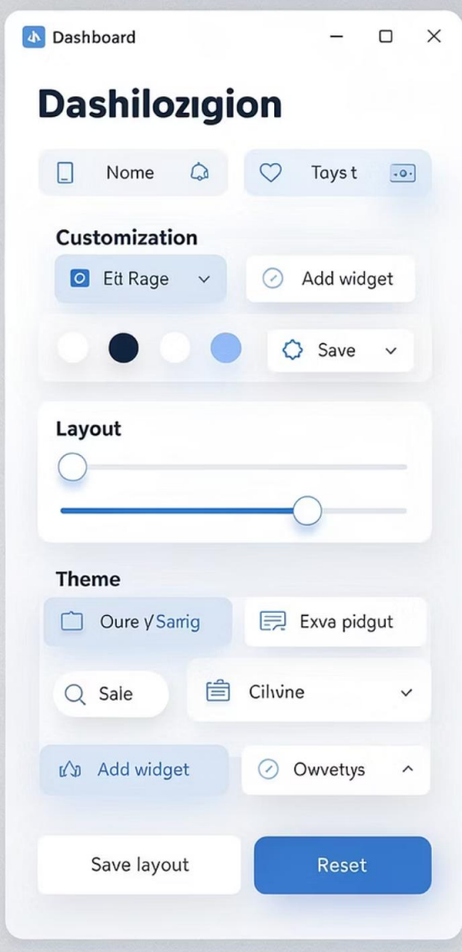
Swing is **built on top of the AWT** — it does not replace it. It uses the same event handling mechanism as the AWT.

AWT vs. Swing: A Comparison

Feature	AWT	Swing
Component Type	Heavyweight (native peers)	Lightweight (pure Java)
Platform Consistency	Varies by OS	Consistent across all platforms
Look and Feel	Fixed (OS-determined)	Pluggable (Metal, Nimbus, custom)
Performance	Faster for simple UIs	Slightly heavier but more flexible
Component Count	~12 basic components	40+ rich components
MVC Support	None	Built-in (Model-Delegate)
Package	java.awt	javax.swing
Introduced	Java 1.0	Java 1.2 (JFC)

Note: Swing does not replace AWT — it is built on top of it. The AWT event model is still used by Swing for all event handling.





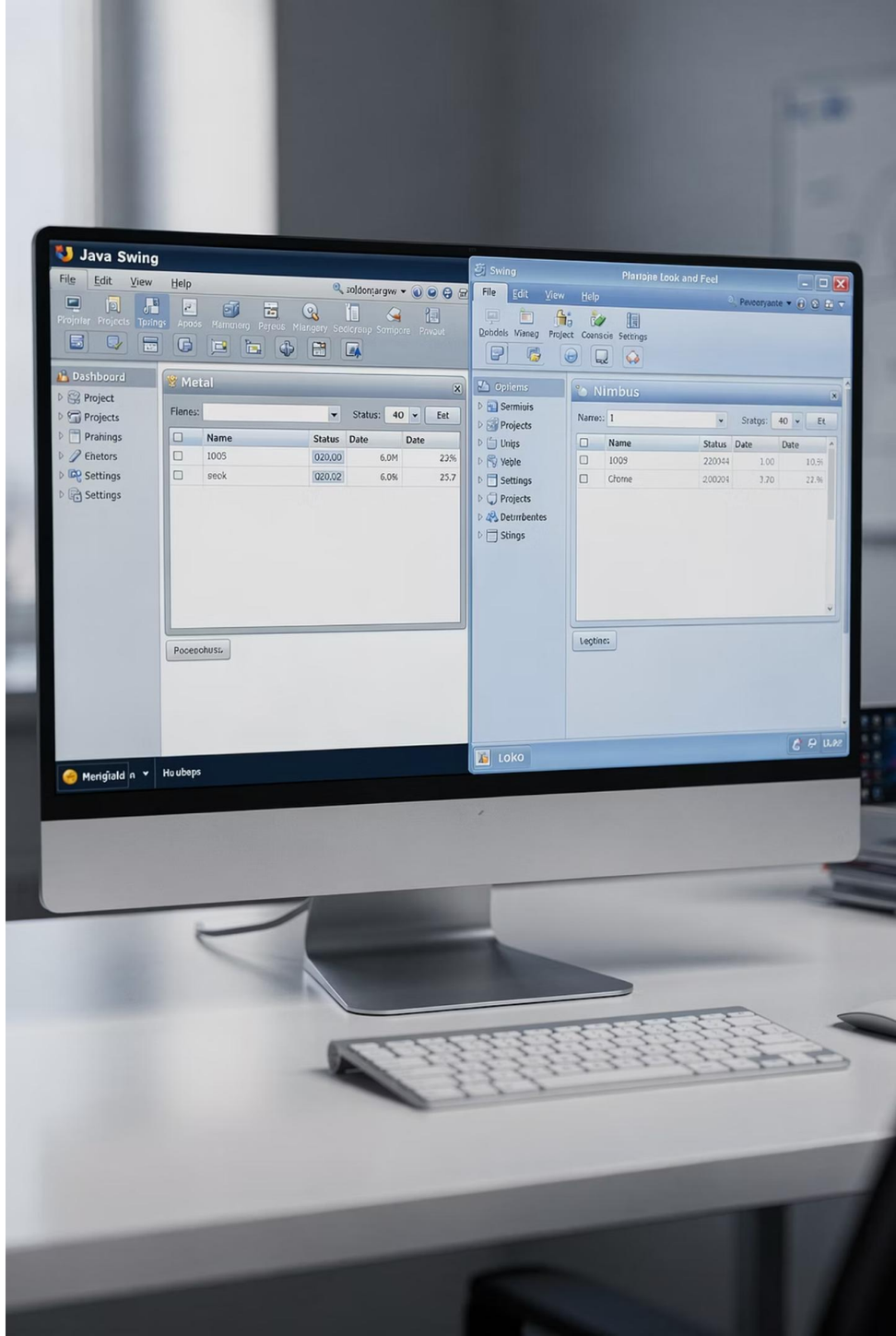
Two Key Swing Features

Lightweight Components

Written entirely in Java — no native peer mapping. More **efficient** and **flexible**. Look and feel is determined by Swing, not the OS, ensuring **consistent behavior across all platforms**.

Pluggable Look and Feel (PLAF)

The look and feel is **separated from component logic**. You can "plug in" a new style without side effects. Java provides built-in look-and-feels: **Metal** (default, platform-independent) and **Nimbus**. Custom look-and-feels can also be designed.



Pluggable Look and Feel in Practice

```
// Set Nimbus Look and Feel
try {
  for (UIManager.LookAndFeelInfo info :
       UIManager.getInstalledLookAndFeels()) {
    if ("Nimbus".equals(info.getName())) {
      UIManager.setLookAndFeel(info.getClassName());
      break;
    }
  }
} catch (Exception e) {
  // Fall back to default
}

// Set System (OS-native) Look and Feel
UIManager.setLookAndFeel(
  UIManager.getSystemLookAndFeelClassName());
```

Available Built-in Look and Feels

- Metal — Default, platform-independent Java look
- Nimbus — Modern, polished Java look (since JDK 6u10)
- System — Matches the host OS (Windows, macOS, GTK)
- Motif — Classic Unix/X11 style

Note: `setLookAndFeel()` must be called before any Swing components are created. Call `SwingUtilities.updateComponentTreeUI(frame)` to apply changes to an existing UI.

Model-View-Controller (MVC)

Every visual component has three aspects: how it **looks**, how it **reacts**, and its **state**. MVC separates these into three parts:

Model

Holds the **state information**. Example: whether a checkbox is checked or unchecked.

View

Determines how the component is **displayed on screen**, reflecting the current model state.

Controller

Determines how the component **reacts to user input**, updating the model accordingly.

Swing's Approach: Swing uses a modified MVC called the **Model-Delegate (Separable Model)** architecture — the View and Controller are combined into a single **UI Delegate**, enabling the pluggable look and feel.

MVC in Action: JButton Example

ButtonModel

Stores the button's state — is it pressed? armed? enabled? selected? The interface is ButtonModel; the default implementation is DefaultButtonModel.

BasicButtonUI

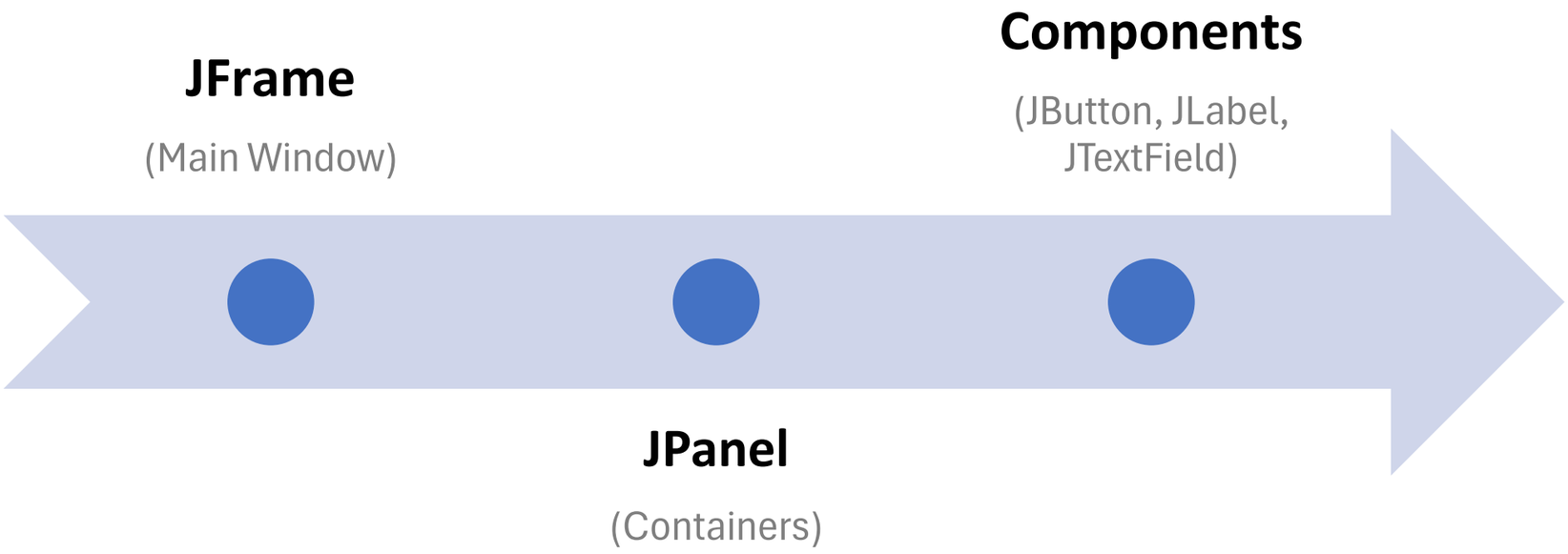
Handles both rendering (how the button looks) and input (mouse clicks, keyboard). The UI Delegate is swapped out when the Look and Feel changes — the model stays the same.

JButton

The facade that ties Model and UI Delegate together. Your code interacts with JButton; it delegates internally to the model and UI delegate.

Note: This is why changing the Look and Feel doesn't affect your application logic — only the UI Delegate is replaced, not the model.

Swing Architecture



Components and Containers

Components

Independent visual controls derived from **JComponent**. All Swing components are in the **javax.swing** package and begin with the letter **J**.

Examples: `JButton`, `JLabel`, `JTextField`, `JCheckBox`, `JList`, `JTable`, `JTree`

Containers

Top-Level (Heavyweight): `JFrame`, `JDialog`, `JWindow`, `JApplet` (deprecated). Must be at the top of every containment hierarchy.

Lightweight: `JPanel` and others that inherit `JComponent`. Used to organize groups of related components.

Top-Level Container Panes: Each top-level container has a **Glass Pane** (topmost, transparent), a **Layered Pane** (manages Z-order), and a **Content Pane** (where you add your components).

Swing Packages

Swing is a large subsystem spanning many packages. Beginning with **JDK 9**, all Swing packages are part of the **java.desktop** module.

javax.swing

Main package — must be imported in every Swing program. Contains core components like buttons, labels, and check boxes.

javax.swing.event

Swing-specific events such as `ListSelectionEvent`, `TreeSelectionEvent`.

javax.swing.plaf.*

Pluggable look-and-feel packages: basic, metal, nimbus, multi, synth.

javax.swing.table / tree / text

Support classes for JTable, JTree, and text components respectively.

Creating First Swing Window

Card showing a complete JList example with event handling.

```
import javax.swing.*;

public class MyFrame {

    public static void main(String[] args) {

        JFrame frame = new JFrame("My First Window");

        frame.setSize(400,300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);

    }
}
```

Key Concepts

`JFrame(name)` — creates a new window
`setSize(width, height)` — sets the window dimensions
`setDefaultCloseOperation()` — tells what to do when user clicks close button
`setVisible(bool)` — changes the visibility of window

A Simple Swing Application

Key Steps in SwingDemo

1 Create a JFrame container

`JFrame jfrm = new JFrame("Title");` — creates a top-level window with title bar and controls.

2 Set size and close operation

`setSize(275, 100)` and `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`

3 Add a JLabel and show

`jfrm.add(jlab)` adds to content pane. `setVisible(true)` displays the window.

4 Use invokeLater()

GUI must be created on the **event dispatching thread** via `SwingUtilities.invokeLater()`.

SwingDemo: Full Code Walkthrough

Card with a full code example of a minimal Swing application, with annotations.

```
import javax.swing.*;
import java.awt.*;

class SwingDemo {
  SwingDemo() {
    // 1. Create top-level container
    JFrame jfrm = new JFrame("Swing Demo");
    jfrm.setSize(275, 100);
    jfrm.setDefaultCloseOperation(
      JFrame.EXIT_ON_CLOSE);

    // 2. Create a label
    JLabel jlab = new JLabel("Swing is powerful!");

    // 3. Add label to content pane
    jfrm.add(jlab);

    // 4. Make the frame visible
    jfrm.setVisible(true);
  }

  public static void main(String[] args) {
    // 5. Run on event dispatching thread
    SwingUtilities.invokeLater(
      () -> new SwingDemo());
  }
}
```

Why Each Step Matters

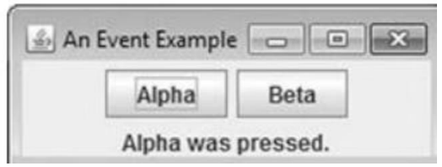
`JFrame.EXIT_ON_CLOSE` — ensures the JVM exits when the window is closed
`jfrm.add(jlab)` — adds to the content pane (not the JFrame directly in older APIs)
`setVisible(true)` — must be called last, after all components are added
`invokeLater()` — Swing is not thread-safe; all GUI creation must happen on the Event Dispatch Thread (EDT)

Event Handling Example: JButton

EventDemo — Key Code

```
JButton jbBtnAlpha = new JButton("Alpha");
jbBtnAlpha.addActionListener(
  new ActionListener() {
    public void actionPerformed(ActionEvent ae) {
      jlab.setText("Alpha was pressed.");
    }
  });

// JDK 8+ Lambda equivalent:
jbBtnAlpha.addActionListener(
  (ae) -> jlab.setText("Alpha was pressed."));
```

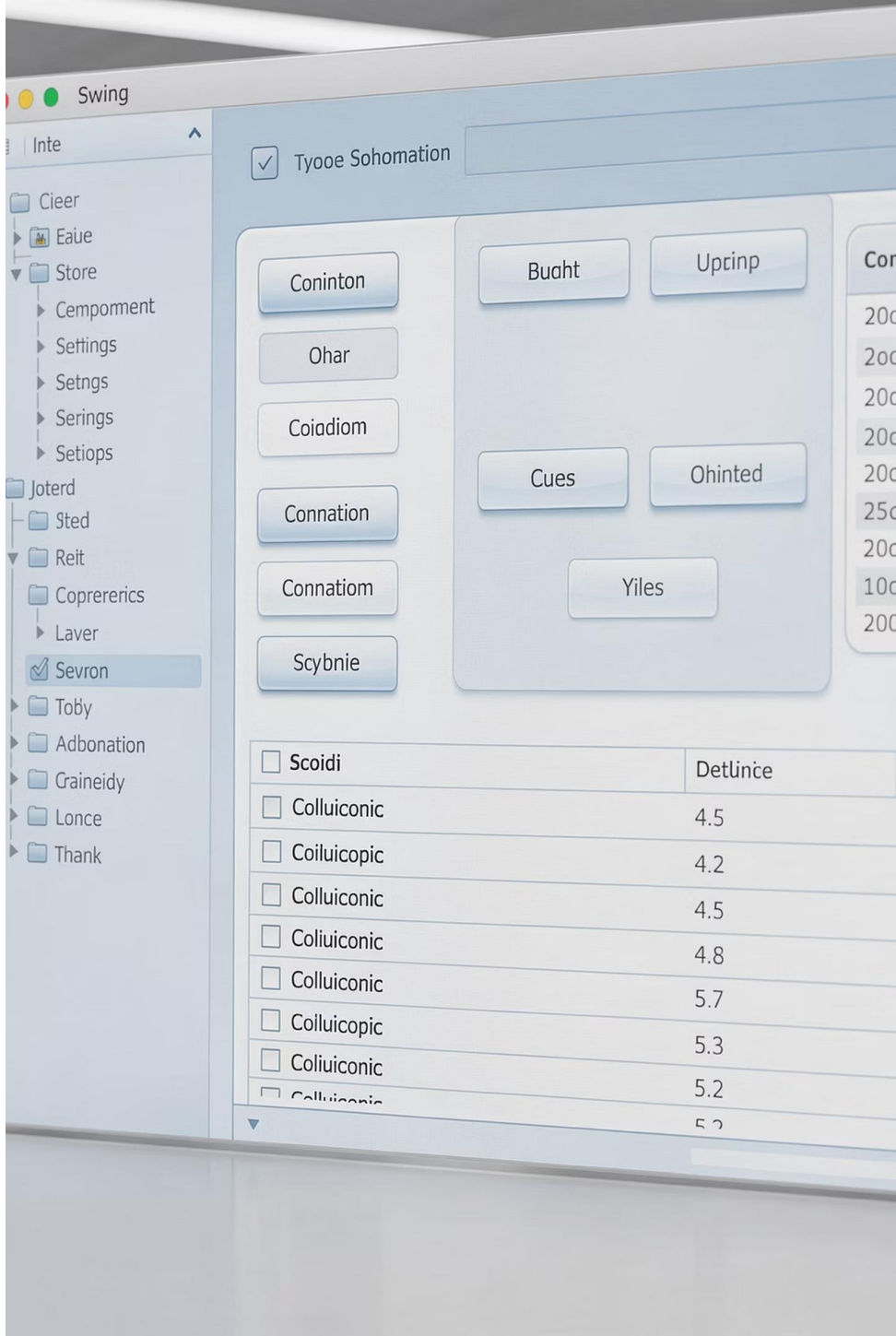


Each button press updates the `JLabel` text. The `getActionCommand()` method identifies which button was pressed when multiple buttons share a listener.

CHAPTER 3

Exploring Swing Components

Swing provides a rich set of lightweight components for building sophisticated GUIs. This section covers the most commonly used components: **JLabel, JTextField, JButton, JToggleButton, JCheckBox, JRadioButton, JTabbedPane, JScrollPane, JList, JComboBox, JTree,** and **JTable**.



JLabel and ImageIcon

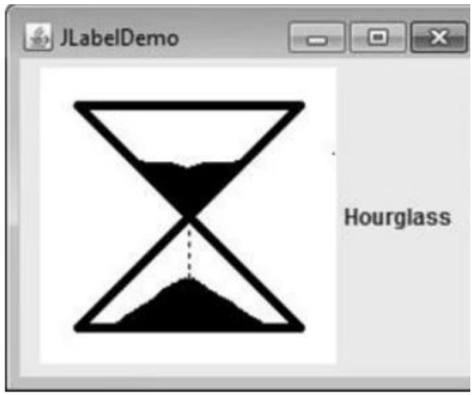
JLabel Overview

Swing's simplest component — passive (no user input). Can display **text**, an **icon**, or both.

Key constructors:

```
JLabel(String str)
JLabel(Icon icon)
JLabel(String str, Icon icon, int align)
```

Use `ImageIcon(String filename)` to load an image. Update dynamically with `setText()` and `setIcon()`.



JLabel: Full Feature Reference

Card expanding on JLabel capabilities beyond the basics.

Text Alignment & HTML Support

JLabel supports HTML content for rich text formatting:

```
JLabel jlab = new JLabel(
  "<html><b>Bold</b> and <i>italic</i><br>" +
  "<font color='red'>Red text</font></html>");
```

Alignment constants:

- SwingConstants.LEFT / CENTER / RIGHT — horizontal
- SwingConstants.TOP / CENTER / BOTTOM — vertical

Set with: `setHorizontalAlignment()` and `setVerticalAlignment()`

Icon Positioning

Control where the icon appears relative to the text:

`setHorizontalTextPosition(SwingConstants.RIGHT)` — text to the right of icon

- `setVerticalTextPosition(SwingConstants.CENTER)`

`setIconTextGap(int gap)` — pixels between icon and text

Key methods summary table:

Method	Purpose
`setText(String)`	Update label text
`setIcon(Icon)`	Update label icon
`setForeground(Color)`	Change text color
`setFont(Font)`	Change font
`setToolTipText(String)`	Add hover tooltip

JTextField

Single-Line Text Input

The most widely used text component. Derived from `JTextComponent`. Uses the `Document` interface as its model.

Key constructors:

```
JTextField(int cols)
JTextField(String str, int cols)
```

Fires **ActionEvent** on Enter key. Use `getText()` to retrieve content. `CaretEvent` fires on cursor movement.

Swing Button Types

All four button types extend **AbstractButton**, sharing common behavior like icon support, rollover icons, and text labels.

JButton

Standard push button. Generates **ActionEvent**. Use `setActionCommand()` to identify buttons in shared listeners.

JToggleButton

Two-state button (on/off). Stays pressed until clicked again. Generates both **ActionEvent** and **ItemEvent**. Superclass of JCheckBox and JRadioButton.

JCheckBox

Independent two-state checkbox. Use `isSelected()` to check state. Generates **ItemEvent** via `ItemListener`.

JRadioButton

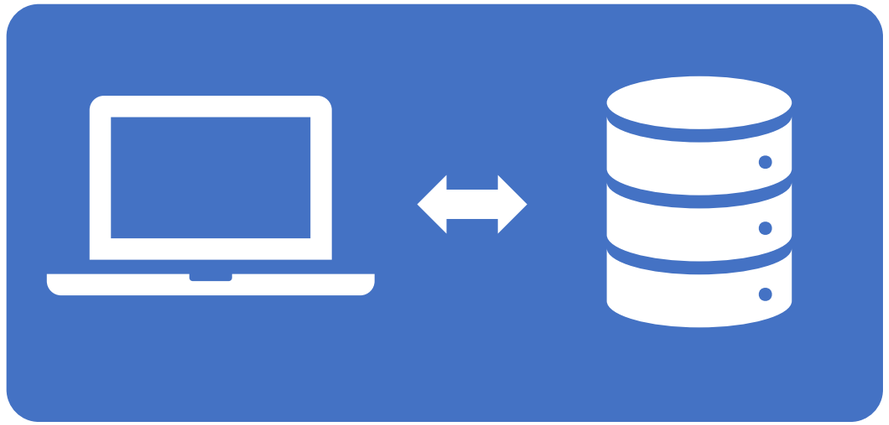
Mutually exclusive selection. Must be added to a **ButtonGroup** to enforce exclusivity. Generates **ActionEvent**.

Database Connectivity with Swing

Swing applications connect to databases using **JDBC (Java Database Connectivity)**. The Swing UI provides the front-end while JDBC handles back-end data operations. Components like `JTable` and `JList` are ideal for displaying query results.

Java Database Connectivity (JDBC)

- JDBC allows Java programs to communicate with databases
- Works with databases like MySQL
- Java sends SQL commands → Database executes them
- Results are returned to Java program



JDBC Architecture

Java Program

↓

JDBC API (java.sql package)

↓

MySQL JDBC Driver

↓

MySQL Database

JDBC Core Classes and Interfaces

Card providing a reference for the key JDBC classes and interfaces used when connecting Swing to a database.

DriverManager

Entry point for JDBC. Use `getConnection(url, user, password)` to obtain a Connection. Automatically loads registered drivers.

Connection

Represents an active database connection. Use `createStatement()` or `prepareStatement()` to create SQL statements. Always close in a finally block or try-with-resources.

Statement

Executes static SQL queries. `executeQuery()` returns a ResultSet. `executeUpdate()` returns the number of affected rows. Vulnerable to SQL injection — prefer PreparedStatement.

PreparedStatement

Pre-compiled SQL with placeholders (`?`). Use `setString()`, `setInt()`, etc. to bind parameters. Safer and faster for repeated queries.

ResultSet

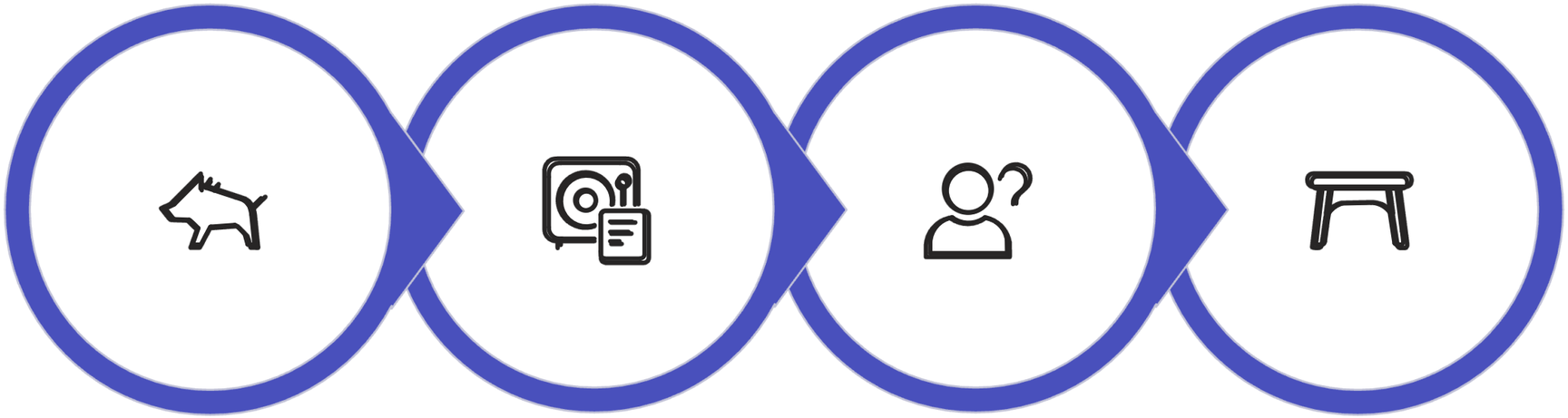
Cursor over query results. `next()` advances to the next row. `getString(col)`, `getInt(col)`, etc. retrieve column values. Always close after use.

SQLException

Checked exception thrown by all JDBC operations. Contains error code, SQL state, and message. Always handle or declare.

Since Java 7, use try-with-resources to automatically close Connection, Statement, and ResultSet objects: `try (Connection con = DriverManager.getConnection(...)) { ... }`

JDBC + Swing: Key Steps



Load JDBC Driver → **Establish Connection** → **Execute SQL Query** → **Display Results**

Data retrieved from the database is typically loaded into arrays or models and then passed to Swing components like `JTable` for display. Always perform database operations on a **background thread** (not the event dispatching thread) to keep the UI responsive.

Steps in JDBC

1. Import package (java.sql.*)
2. Load JDBC Driver
3. Create Connection
4. Create Statement
5. Execute SQL Query
6. Process ResultSet
7. Close Connection

JDBC Integration Example

Connecting to a Database

```
// Load driver
Class.forName("com.mysql.jdbc.Driver");

// Get connection
Connection con = DriverManager.getConnection(
  "jdbc:mysql://localhost:3306/mydb",
  "user", "password");

// Execute query
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery(
  "SELECT * FROM employees");
```

Displaying in JTable

```
// Build data arrays from ResultSet
ArrayList<String[]> rows = new ArrayList<>();
while(rs.next()) {
  rows.add(new String[]{
    rs.getString("name"),
    rs.getString("ext"),
    rs.getString("id")
  });
}
// Pass to JTable
JTable table = new JTable(
  data, colHeads);
jfrm.add(new JScrollPane(table));
```

Use `SwingWorker` for background DB operations to avoid blocking the UI thread.

CRUD Operations

- **CREATE** → INSERT INTO student VALUES(...)
- **READ** → SELECT * FROM student
- **UPDATE** → UPDATE student SET ...
- **DELETE** → DELETE FROM student WHERE ...